

Pramo Jewels

Software Architecture Document

Document ID: PJ-SWARCH-001
Version: 1.0
Status: Draft

Field	Value
Project	Pramo Jewels Premium E-Commerce Platform
Purpose	Define the internal software architecture, layers, modules and design patterns
Audience	Software Architects, Backend & Frontend Developers, QA, AI Coding Agents
Version	1.0

1. Architectural Style

Adopt a modular layered architecture separating presentation, application, domain and infrastructure concerns. Components communicate through well-defined REST APIs and service interfaces.

2. Layered Architecture

Presentation Layer: Next.js UI, Admin Portal.
Application Layer: Controllers, Request Validation, Service Orchestration.
Domain Layer: Business Rules, Order Processing, Pricing, Inventory.
Infrastructure Layer: Database, Cache, Storage, External APIs.

3. Major Software Modules

Authentication & Users
Product Catalogue
Search & Filters
Wishlist & Cart
Checkout
Payments
Orders & Returns
Inventory
Courier Integration
Notifications
Reports & Analytics
Administration

4. Design Patterns

Repository Pattern
Service Layer Pattern
Dependency Injection
Factory Pattern
Strategy Pattern (Payments/Shipping)
Observer/Event Pattern (Notifications)

Middleware Pattern (Security & Authorization)

5. API Communication

Frontend communicates with backend through versioned REST APIs returning standardized JSON responses. External integrations are isolated through adapter services.

6. Data Management

Use MySQL as the system of record, Redis for caching and sessions, and object storage for product images and media. Transactions protect order and payment consistency.

7. Error Handling

Centralized exception handling
Consistent API error format
Validation responses
Audit logging for critical failures
Graceful fallback for external service outages

8. Security by Design

Authentication via Laravel Sanctum/JWT
Role-Based Access Control
Input validation and sanitization
Encryption of sensitive data
Secure secret management

9. Scalability & Maintainability

Keep modules loosely coupled, follow SOLID principles, encapsulate business logic in services, and support future migration to microservices if required.

10. Quality Attributes

Performance
Reliability
Maintainability
Extensibility
Security
Testability
Observability

11. Testing Strategy

Unit tests for business logic, integration tests for APIs, end-to-end tests for user journeys, and regression testing before production releases.

12. Future Evolution

Architecture should support mobile applications, AI-powered recommendations, multilingual content, loyalty programs, and additional sales channels without major redesign.